

Assignment - 05Tutorial - Sheet 09Section - A

1. Define a graph and differentiate between directed & undirected graphs.

Ans: A graph is a collection of vertices (nodes) connected by edges (links).

- Directed Graph: Edges have a specific direction ($A \rightarrow B$).
- Undirected Graph: Edges have no direction, representing mutual connection ($A-B$).

2. Difference between adjacency matrix and adjacency list representations of a graph?

Ans: Adjacency Matrix: Uses an $N \times N$ matrix. 1 indicates an edge, 0 indicates no edge. Fast lookup but uses more memory.

Adjacency List: Stores edges as lists for each vertex; saves memory but lookup is slower.

3. Define degree, path and cycle in a graph with examples.

Ans: Degree: Number of edges incident on a vertex. Example: If A connects to B and C, $\deg(A) = 2$.

Path: Sequence of edges connecting vertices. Eg: $A \rightarrow B \rightarrow C$

Cycle: Path that starts & ends at the same vertex. Eg: $A \rightarrow B \rightarrow C \rightarrow A$

4. Mention any two real-life applications of graphs.

Ans: Social networks (users as nodes, connections as edges).

Navigation systems like Google maps (locations as nodes, roads as edges).

5. What data structure is used in DFS and BFS respectively.

Ans: DFS: Uses a stack (explicit stack or recursion).

BFS: Uses a queue.

Section-B

1. What is a minimum spanning tree? Explain how Kruskal's Algorithm constructs it with an example.

Ans: A Minimum Spanning Tree (MST) of a connected, weighted, undirected graph is a subset of edges that connects all vertices with no cycles and with the minimum total weight. An MST always contains $(V-1)$ edges.

Kruskal's Algorithm is a greedy technique that builds the MST by always choosing the smallest available edge that does not form a cycle.

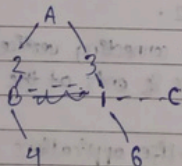
Steps:

- (i) List all edges of the graph.
- (ii) Sort them in non-decreasing order of the weight.
- (iii) Start with an empty MST.
- (iv) Pick the smallest edge.
 - If adding it does not form a cycle, include it.
 - Otherwise, skip it.

(v) Continue until MST has $V-1$ edges.

Cycle checking is done using Disjoint Set (Union-Find).

Example: Consider a graph with edges:



Edges Sorted:

1(B-C), 2(A-B), 3(A-C), 4(B-D), 5(D-E), 6(C-E)

Construction:

- Pick B-C (1) $\rightarrow$ MST = {B-C}
 - Pick A-B (2) $\rightarrow$ MST = {B-C, A-B}
 - Skip A-C (3) (creates cycle)
 - Pick B-D (4) $\rightarrow$ MST = {B-C, A-B, B-D}
 - Pick D-E (5) $\rightarrow$ MST = {B-C, A-B, B-D, D-E}
- All 5 vertices connected $\rightarrow$ MST complete.
Final MST weight = $1 + 2 + 4 + 5 = 12$.

2. Write the algorithm for Prim's Algorithm & trace it for the given weighted graph.

Ans: Prim's Algorithm:

- (i) Start from any vertex (say A).
- (ii) Maintain a set S of vertices included in MST (initially {start}).
- (iii) Repeatedly add the minimum-weight edge that has one endpoint in S & the other outside S .
- (iv) Continue until all vertices are included.

Typical Implementation: Use a min-priority queue keyed by cheapest edge to each outside vertex (complexity $O(E+V \log V)$ with binary heap or $O(E \log V)$).

Trace on the same graph (vertices A, B, C, D, E and edges as above):

- Start with A. $S = \{A\}$. Candidate edges: A-B(2), A-C(3). Pick A-B(2) $\rightarrow S = \{A, B\}$.
- Candidates (edges from S to outside): A-C(3), B-C(1), B-D(4). Pick B-C(1) $\rightarrow S = \{A, B, C\}$.
- Candidates: B-D(4), C-D(5), C-E(6). Pick B-D(4) $\rightarrow S = \{A, B, C, D\}$.
- Candidates: C-E(6), D-E(5). Pick C-E(6) $\rightarrow S = \{A, B, C, D, E\}$.
- Stop - MST edges chosen: A-B(2), B-C(1), B-D(4), C-E(6). Same MST & total weight = 13.

When Prim is useful - dense graphs, or when you want to grow a single connected tree starting from a 'root'. Also simpler with adjacency lists + heap.

3. Explain Warshall's Algorithm and compute the transitive closure for the given adjacency matrix.

Ans: Warshall's Algorithm is used to compute the transitive closure of a directed graph. It determines for every pair (i, j) whether there exists a path from vertex i to vertex j .

Goal = If there is a path from vertex $i \rightarrow j$, then

$$R[i][j] = 1$$

Algorithm/ Pseudo Code:

Input: $n \times n$ adjacency matrix T ($T[i][j] = 1$ if edge $i \rightarrow j$ else 0)

for $k = 1$ to n :

for $i = 1$ to n :

for $j = 1$ to n :

$T[i][j] = T[i][j] \text{ OR } (T[i][k] \text{ AND } T[k][j])$

OUTPUT: T (transitive closure)

Time Complexity: $O(n^3)$; Space Complexity: $O(n^2)$

Example: Adjacency matrix for 4 vertices: $(1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \text{ chain})$.

$$T_0 = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Apply Warshall -

- $k=1$: No new paths.
- $k=2$: $T[1][3] = T[1][3] \text{ OR } (T[1][2] \text{ AND } T[2][3]) = 1 \rightarrow$ reaches 3.
- $k=3$: $T[1][4] = 1$ and $T[2][4] = 1$ via vertex 3.
- $k=4$: no change.

$$T^* = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Section - C

Compare and contrast DFS & BFS in terms of data structure used, time complexity, and applications. Also, write pseudo for both.

Ans: Data Structure Used:

- DFS = Uses a stack.
- BFS = Uses a queue.

Time Complexity:

- DFS = $O(V+E)$
- BFS = $O(V+E)$

Applications:

DFS -

- detecting cycles in graphs.
- Solving maze & puzzle problems.
- Topological sorting.
- Finding connected components.
- Path finding in deep graphs.

BFS -

- Finding shortest paths in an unweighted graph.
- Level-order traversal for trees.
- Searching in layers.
- Networking finding broadcast paths.
- Finding minimum no. of moves.

Pseudo Code for DFS:

* DFS (Recursive version)

DFS(G, start):

mark start as visited

for each neighbour V of start:

If V is not visited:
 $DFS(G, V)$

* DFS (Using stack)

DFS($G, start$):

Create a stack S

push $start$ on S

mark $start$ as visited

while S is not empty:

node = pop

for each neighbor V of node:

If V is not visited:

mark V as visited

push V on S .

Pseudo code for BFS:

BFS($G, start$):

Create a queue Q

Enqueue $start$

mark $start$ as visited

while Q is not empty:

node = dequeue Q

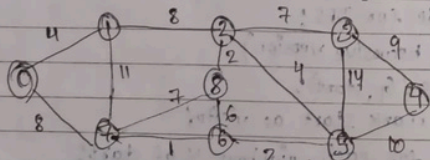
for each neighbour V of node:

If V is not visited:

mark V as visited

enqueue V .

2: Apply Dijkstra's Algorithm on the following weighted graph and show



Ans: It is used to find shortest path distance in a weighted graph with non-negative weights.

At $0 \rightarrow 1$ Need Distance

$$0 + 4 = 4$$

$$0 + 7 = 7$$

At $0 \rightarrow 2$

$$d[2] = 4 + 8 = 12$$

At $0 \rightarrow 6$

$$d[6] = 8 + 1 = 9$$

At $0 \rightarrow 5$

$$d[5] = 8 + 1 + 2 = 11$$

$$d[5] = 4 + 8 + 4 = 16$$

$$d[5] = 8 + 1 + 6 + 2 + 4 = 21$$

At $0 \rightarrow 3$

$$d[3] = 4 + 8 + 7 = 15$$

At $0 \rightarrow 4$

$$d[4] = 8 + 1 + 2 + 10 = 21$$

$$d[4] = 4 + 8 + 7 + 9 = 28$$

Final shortest distance path:

$$d[0] = 0$$

$$d[8] = 4$$

$$d[1] = 4$$

$$d[2] = 12$$

$$d[3] = 19$$

$$d[4] = 21$$

$$d[5] = 11$$

$$d[6] = 9$$

$$d[7] = 4$$

3. Explain the working of Marshall's Algorithm in detail, derive the formula used for updating the reachability matrix and illustrate the algorithm with an example graph.

Ans: Computes the transitive closure of a directed graph for every pair of vertices i, j decide whether there exists any path from i to j .
Notation & Derivation:

Let $T^k[i][j]$ be a Boolean value meaning "there is a path from i to j whose intermediate vertices (if any) are only from the set $\{1, 2, \dots, k\}$ ".

Base: $T^0 =$ adjacency matrix (direct edges only).

When we allow vertex k as an additional intermediate, a path $i \rightarrow j$ either:

- Already exists without using k (so $T^{(k-1)}[i][j] = 1$) or
 - can be formed by going $i \rightarrow k$ & $k \rightarrow j$ using vertices $\leq k-1$ inside those subpaths (so $T^{(k-1)}[i][k] \wedge T^{(k-1)}[k][j] = 1$).
- Thus, the recurrence is:

$$T^{(k)}[i][j] = T^{(k-1)}[i][j] \vee (T^{(k-1)}[i][k] \wedge T^{(k-1)}[k][j])$$

In-place implementation uses the simplified update:

for $k = 1 \dots n$:

for $i = 1 \dots n$:

for $j = 1 \dots n$:

$$T[i][j] = T[i][j] \text{ OR } (T[i][k] \text{ AND } T[k][j])$$

Time complexity: $O(n^3)$ & Space: $O(n^2)$

Example: Graph vertices 1, 2, 3, 4 with directed edges 1 $\rightarrow$ 2, 2 $\rightarrow$ 3, 3 $\rightarrow$ 4

Initial adjacency T^0 :

0	1	0	0
0	0	1	0
1	0	0	1
0	0	0	0

Apply Warshall (showing matrix after each k):

- After $k=1$ (allow intermediate $\{1\}$):

$$T^{(1)} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

After $k=2$

$$T^{(2)} = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

After $k=3$

$$T^{(3)} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

After $k=4$

$$T^{(4)} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$